VAREN MET JE SMARTPHONE

NHL HOGESCHOOL

Drie Handige Apps Die De Watersporter Van Informatie Voorzien

Eddie Sprietsma
Jasper Stein
Mathijs Jonker
Marten Schilstra

Projectgroep #27

VAREN MET JE SMARTPHONE

Drie Handige Apps Die De Watersporter Van Informatie Voorzien

Project P4P
Software Engineering
NHL Hogeschool

Opdrachtgevers
NHL Internet Academie
Frederik van der Meulen
Siger Bos

Tutor
Greta Schaafsma

Datum
14 Juni 2012

Eddie Sprietsma
Jasper Stein
Mathijs Jonker
Marten Schilstra

Projectgroep #27

Inhoud

Samenvatting....4
1 Inleiding....5
2 De Opdracht....7
2.1 Wat houdt de opdracht in?....7
2.2 Waarom wil de opdrachtgever dit?....8
3 Het onderzoek....9
4 De applicaties....11
4.1 Hoe en waarmee worden de applicaties gemaakt?....11
4.2.1 De ‘main-app’....12
4.2.2 Bruggen en Aanlegplaatsen app....13
4.2.3 Bezienswaardigheden Leeuwarden app....14
4.2.4 Visstekken app....15
5 Het omzetten van data....16
6 Verloop van het project....18
7 Samenwerking....19
8 Conclusies....20
9.1. Design appicoon....21
9.2 Design iconen in de app....21
9.3 Titanium Studio....21
Verklarende Lijst....23
Bronnenlijst....25
Bijlagen....26
A Keuze ontwikkelplatform voor water apps....26

Samenvatting

Dit rapport bevat het resultaat van drie watersport apps die ontwikkeld zijn voor de watersporters in Friesland in opdracht van de NHL Internet Academie.

Het rapport is geschreven in het kader van een project voor de opleiding Informatica op de NHL Hogeschool. In de vierde periode van deze opleiding moet er een project gedaan worden, waarin je als groep een opdracht van een bedrijf of instantie uitvoert.

De opdrachtgever is van mening dat de watersporters op het water in Friesland nu te weinig informatie hebben om bijvoorbeeld te weten of er onder een bepaalde brug door kan worden gevaren of niet, of er aanlegplaatsen in de buurt zijn om aan te leggen of dat er leuke bezienswaardigheden of restaurantjes in de buurt zijn.

De opdrachtgever wil graag zien dat er drie apps ontwikkeld worden, deze drie apps hebben vergelijkbare functionaliteit, maar verschillen van thema. Het gaat om de volgende drie thema's:

1. **Bruggen en Aanlegplaatsen**: Geeft informatie over bruggen en aanlegplaatsen in heel Friesland.
2. **Bezienswaardigheden Leeuwarden**: Geeft informatie over bezienswaardigheden aan het water, bruggen en aanlegplaatsen in Leeuwarden en omgeving.
3. **Visstek app**: Geeft informatie over waar visstekken in Friesland te vinden zijn.

Hier volgt een korte beschrijving van hoe het ontwikkelproces van de drie apps is verlopen. Als eerste is er gedocumenteerd wat een gebruiker met de app wil kunnen en aan de hand daarvan is beschreven hoe de app moet gaan werken, vervolgens zijn er aan de hand van deze informatie *mockups* van de apps gemaakt. Daarnaast is er ook onderzocht waarmee de apps het beste ontwikkeld kunnen worden. Nadat de opdrachtgever dit voorafgaand onderzoek heeft goedkegeurd, is het programmeer werk van de apps begonnen, eerst is alle gemeenschappelijke functionaliteit van de apps ontwikkeld en vervolgens nog de functionaliteit die bij een specifiek thema toebehoort. De laatste fase van het ontwikkelproces is de acceptatietest door de opdrachtgever en het eventueel nog aanpassen van de apps uit feedback daarvan.

De apps zijn ontwikkeld voor *smartphone*s en *tablets* met het *Android* of *iOS* besturingssysteem. De applicaties zijn ontwikkeld met het Titanium appcelerator software ontwikkel pakket. Er wordt afgeraden om Titanium appcelerator te gaan gebruiken voor een vergelijkbare opdracht, omdat het soms de nodige functionaliteit mist of niet goed werkt en ook niet krachtig genoeg is.

De *Android* versie van de apps worden in de Google Play Store geplaatst, waarmee je deze apps gratis kunt installeren op je *Android smartphone*. Met *iOS* versie wordt voorlopig niets gedaan.

1 Inleiding

In de vierde periode van het eerste jaar van de opleiding Informatica aan de NHL hogeschool moeten de studenten verdeeld in projectgroepen een opdracht uitvoeren van een bestaande instantie of bedrijf. Ondergetekende groepje heeft als opdrachtgever de NHL Internet Academie.

De NHL Internet Academie is van mening dat de watersporters op het water in Friesland nu te weinig informatie hebben om bijvoorbeeld te weten of er onder een bepaalde brug door kan worden gevaren of niet, of er aanlegplaatsen in de buurt zijn om aan te leggen of dat er leuke bezienswaardigheden of restaurantjes in de buurt zijn.

De opdrachtgever wil graag zien dat er drie apps ontwikkeld worden, deze drie apps hebben vergelijkbare functionaliteit, maar verschillen van thema. Het gaat om de volgende drie thema's:

1. **Bruggen en Aanlegplaatsen**: Geeft informatie over bruggen en aanlegplaatsen in heel Friesland.
2. **Bezienswaardigheden Leeuwarden**: Geeft informatie over bezienswaardigheden aan het water, bruggen en aanlegplaatsen in Leeuwarden en omgeving.
3. **Visstek app**: Geeft informatie over waar visstekken in Friesland te vinden zijn.

Verder wil de opdrachtgever dat de applicaties voldoen aan de volgende eisen:

- De applicatie moet gebruiksvriendelijk en overzichtelijk zijn.
- De applicatie moet op *iOS* en *Android* werken, zowel op de *smartphone* als op de *tablet*.
- De applicatie moet te gebruiken zijn op het water, dus zowel offline als online moet de app kunnen functioneren.
- De applicatie moet een kaart bevatten.
    - Op deze kaart moeten de bezienswaardigheden van de desbetreffend a*pp* worden getoond.
    - Op deze kaart moet de huidige positie van de gebruiker getoond worden.
    - Op deze kaart moet een *trail* van de gebruiker getoond worden.
- De applicatie moet een duidelijke weergave hebben van de in de desbetreffende a*pp* belangrijke bezienswaardigheden in de vorm van een lijst.
- In de applicatie, moet je kunnen zoeken op bezienswaardigheden van de desbreffende *app*.
- De applicatie moet informatie geven over de bezienswaardigheden.
- De applicatie moet rekening houden met de hoogte en de breedte van de boot.
- De code van de applicatie moet goed gedocumenteerd worden.
- De applicatie moet makkelijk te onderhouden zijn.

Voordat de apps daadwerkelijk gemaakt worden moet er een klein onderzoek worden gedaan, de hoofdzaak in dat onderzoek is om er achter te komen hoe er het beste aan de eisen van de opdrachtgever kan worden voldaan. Het onderzoek kan daarnaast nog verder worden verdeeld in de volgende deelvragen:

- Wat maakt een app gebruiksvriendelijk?
- Wat zijn de verschillen tussen *iOS* en A*ndroid*?
    - Hoe krijg je de applicatie goed werkend op beide platformen?
- In welke ontwikkelomgeving kunnen de apps het beste worden gemaakt?
- Hoe voeg je een kaart toe aan een applicatie?
    - Wat is de beste manier om deze kaart zowel *offline* als *online* te gebruiken?
    - Hoe krijg je de punten overzichtelijk op de kaart?
    - Hoe vind je de huidige positie van de gebruiker op de kaart?
        - Hoe bewaar je deze tijdelijk voor een *trail*?
    - Met welke straal op de kaart is standaard het handigste te gebruiken?

Nadat het onderzoek is gedaan en goedgekeurd is door de opdrachtgever, kunnen de apps worden gemaakt aan de hand van de resultaten van het onderzoek. Omdat de drie apps veel vergelijkbare functionaliteit heeft, kan deze eerst worden gemaakt om vervolgens daarna de drie aparte thema apps te maken.

2 De Opdracht

Dit hoofdstuk beschrijft de opdracht van de opdrachtgever in detail. Wat houdt de opdracht in? Waarom wil de opdrachtgever dit?

2.1 Wat houdt de opdracht in?

Het is de bedoeling om voor de NHL Internet Academie drie apps te gaan maken die elk iets te maken hebben met de wateren in Friesland. De één heeft als thema de bruggen en aanlegplaatsen in Friesland weer te geven, de ander geeft de bezienswaardigheden in Leeuwarden langs het water weer en de derde geeft de visstekken in Friesland weer.

De apps die gemaakt worden zijn bedoeld voor *smartphone*s en *tablet*s met het *Android* of *iOs* besturingssysteem, dit zijn namelijk gebruiksvoorwerpen die men tegenwoordig vaak mee neemt wanneer men er met de boot op uit gaat.

De apps moeten natuurlijk voor de gebruiker niet verwarrend zijn, dus de apps moeten makkelijk te bedienen zijn en alle functionaliteit moet gemakkelijk ontdekt kunnen worden. Het gaat vooral om de bezienswaardigheden die de gebruikers in de applicaties kunnen ontdekken, dus deze moeten goed in beeld worden gebracht. De opdrachtgever wil graag een kaart in de app hebben waarop de bezienswaardigheden ontdekt kunnen worden, maar ook een lijst waarin alle bezienswaardigheden opgesomd in staan, ook moet de gebruiker naar een bepaalde bezienswaardigheid kunnen zoeken. Voor elke bezienswaardigheid geld ook dat er een detail scherm voor moet zijn die gedetaileerde informatie over de bezienswaardigheid weergeeft.

Ook moet de gebruiker zijn eigen positie makkelijk terug kunnen vinden op de eerder genoemde kaart. Daarbij zou het mooi zijn als er een *trail* bij de locatie word aangemaakt, zodat de gebruiker kan zien in welke richting hij vaart.

In het speciale geval van bruggen moet er ook een manier zijn om aan te geven dat de gebruiker met de aangegeven afmetingen van zijn boot niet door een brug kan.

Daarnaast is het ook zo dat de gebruiker de applicatie gebruikt op plekken waar niet altijd een goede internet verbinding is, dus de applicaties moeten zo veel mogelijk onafhankelijk zijn van een internet verbinding.

Nadat het project voltooid is moet ook een andere ontwikkelaar gemakkelijk met de geschreven applicaties aan de slag, dus de code moet makkelijk te onderhouden en goed gedocumenteerd zijn.

2.2 Waarom wil de opdrachtgever dit?

De NHL Internet Academie vindt dat watersporters niet genoeg informatie hebben op het water. Wanneer er op het water wordt gevaren, is er de behoefte om te weten wat voor bruggen er op de route liggen en of er onderdoor gevaren kan worden. Ook is er de behoefte om te weten waar aanlegplaatsen zich bevinden, zodat men snel en gemakkelijk ergens aan kan leggen.

Ook is er nu nog niks, wanneer men een boottocht in Leeuwarden maakt, om te kijken waar er zich bezienswaardigheden, bruggen of aanlegplaatsen bevinden. Als een onbekende in Leeuwarden komt met de boot, wil die graag weten of er zich bruggen op de route begeven waar de boot niet onderdoor kan, of er aanlegplaatsen in de buurt liggen en of er leuke plekjes zijn waar er wat gegeten of gedronken kan worden.

Tenslotte heeft de NHL Internet Academie bevonden dat er nog geen app is voor vissers die niet goed weten waar visstekken zich bevinden. Vissers hebben behoefte aan een overzicht van visstekken in de buurt, maar deze worden digitaal nog niet goed weergegeven, hierdoor weten vissers vaak niet waar goede visstekken zich bevinden.

Daarom wil de opdrachtgever graag dat de drie applicaties gemaakt worden, want zowel in de *App Store* als in de *Android Play Store* zijn er nog geen programma's voor de mobiele telefoon die aan de eisen van de opdrachtgever voldoen.

3 Het onderzoek

Dit hoofdstuk verdiept zich in de inleiding gestelde onderzoeksvragen.

Om een *app* gebruiksvriendelijk te maken zijn er een aantal zaken belangrijk om mee te nemen in het ontwikkelproces: *[1][2]*

1. ***apps* afsluiten:** Zorg ervoor dat de gebruiker de *app* altijd en gemakkelijk kan afsluiten, zonder daar een taakmanager voor nodig te hebben.
2. **Gegevens opslaan:** Zorg ervoor dat de gegevens van de *app* zo weinig mogelijk het interne geheugen belasten. Wanneer er een extern geheugen beschikbaar is zou dit de eerste optie moeten zijn.
3. **Overzichtelijk:** Maak de navigatie van de *app* niet moeilijker dan nodig is. Complexe navigatie schrikt gebruikers af. Tabbladen zijn in veel gevallen praktisch en intuïtief.
4. **Snelheid:** De mobiele gebruiker is (vaak) jong, ongeduldig en niet gewend om langer dan twintig seconden te wachten. Kijk goed na hoe snel de a*pp* is.
5. **Achtergrondprocessen:** Zorg ervoor dat de *app* niet veel energie verbruikt, zodra deze op de achtergrond nog actief is.
6. **Volledig benutten:** Probeer zoveel mogelijk de mogelijkheden van de desbetreffende telefoon te benutten.
7. **Feedback:** De *app* moet de gebruiker voldoende feedback geven.
8. **Bediening:** De bediening van de *app* moet simpel zijn, zodat de gebruiker het snel kan aanleren.

De verschillen tussen *iOS* en *Android* zijn essentieel om te bestuderen, omdat er hiermee tijdens het ontwikkelen van de *apps* rekening gehouden moet worden. Dit zijn de grootste verschillen: *[3]*

Afbeelding 2: Het logo van Apple

Afbeelding 1: Het logo van Android

iOS:

1. Één soort telefoon, de *iPhone*, één soort *tablet*, de *iPad* en éénzelfde soort muziekspeler, de *iPod Touch*. Allemaal van hetzelfde merk, apple. Deze zijn er allemaal in verschillende versies, alleen is de basis overal hetzelfde.
2. Heeft geen mogelijkheid om zelf je hele telefoon of *tablet* te personaliseren. Het zijn vooraf ingestelde instellingen die je zelf, tot op een zekere hoogte, kunt wijzigen.
3. Heeft maar één fysieke knop op het scherm, de 'homeknop', een paar geluidsknoppen aan de linkerkant, een 'sleepknop' op de bovenkant en voor de rest alleen een touchscherm.

Android:

1. Meerdere soorten telefoons en *tablet*s van verschillende merken. Hierdoor moet er rekening gehouden worden met de verschillende groottes en capaciteiten.
2. Heeft eindeloze mogelijkheden om zelf je hele telefoon of *tablet* te personaliseren.
3. De mogelijkheid om *widgets* op je startscherm te zetten.

De informatie voor op het water applicatie wordt goed werkend op beide platformen, door een goede ontwikkelomgeving te kiezen, waarin er gemakkelijk onderscheid kan worden gemaakt tussen *iOS* en *Android*.

Er is grondig onderzoek gedaan naar de verschillen tussen ontwikkelomgevingen, waarin de *apps* kunnen worden ontwikkeld *(Bijlage A, p.18)*. Uiteindelijk is er gekozen voor *Titanium Studio*. Het grootste voordeel hiervan is, dat je snel en *cross platform*, *native apps* kunt ontwikkelen, met een populaire programmeertaal, genaamd *JavaScript. [4]*

De beste manier om een kaart in de *app* te krijgen is via *Google Maps*. Deze kaart is *online* te gebruiken door middel van mobiel internet, *Wi-Fi* en *GPS*. Ook is deze kaart *offline* te gebruiken, want hij *cached* het gedeelte van de kaart waar de gebruiker op dat moment naar kijkt. *[5]*

Om de punten op de kaart te krijgen, zijn er de standaard ingebouwde *annotations* van *Google Maps*.
Om ze overzichtelijk op een scherm te laten zien is er de mogelijkheid, zodra je *uitzoomt* op een kaart om de *annotations* te groeperen in hoeveelheden. *[6]*

Afbeelding 3: Google Maps

De huidige positie wordt bepaald door middel van *GPS*, hiermee wordt via satellieten de huidige positie bepaald. Om hierna een *trail* op de kaart te tonen is niet lastig. De huidige positie wordt onthouden en zodra deze zich verplaatst wordt er een stip op de kaart getoond die de *trail* vormt. *[7]*.

De straal die het beste kan worden gebruikt voor het tonen van de kaart, zodra de gebruiker de app opent is de gehele regio waar het om gaat, mits er geen *GPS* verbinding is.

4 De applicaties

In dit hoofdstuk wordt beschreven hoe de applicaties zijn gemaakt en hoe ze werken.

Er worden drie *apps* gemaakt die informatie geven op en rond het water in Friesland. De *apps* hebben hetzelfde raamwerk, waar de functionaliteit van het eigen thema aan toegevoegd word. De drie *apps* die moeten worden ontwikkeld zijn:

1. **Bruggen en Aanlegplaatsen app**: Wanneer er op het water wordt gevaren wil men graag weten of er ook bruggen zijn die men tegen gaat komen, zodat er rekening gehouden kan worden met de hoogte van de brug. Ook wil men graag weten of er aanlegplaatsen in de buurt zijn, waar men met de boot kan aanleggen.
2. **Bezienswaardigheden Leeuwarden app**: Zodra men in Leeuwarden komt met de boot, wil men graag weten wat voor bruggen, aanlegplaatsen en bezienswaardigheden er in de buurt zijn.
3. **Visstekken app**: Voor vissers is het voornamelijk handig om te weten waar er zich goede visstekken bevinden om goed te kunnen vissen

Alle drie de *apps* moeten op twee platformen beschikbaar zijn:

1. *Android*
2. *iOS*

Voor zowel *Android* als *iOS* moet het op de *smartphone* en de *tablet* versie werken.

Verder moeten de *apps* informatie geven over de bruggen, aanlegplaatsen en visstekken en bij Leeuwarden alle gegevens over de POI's in de buurt. Ten slotte is het ook nog van groot belang dat de *apps* ook op het water goed blijven werken.

4.1 Hoe en waarmee worden de applicaties gemaakt?

Eerst wordt er algemeen verteld hoe er wordt gewerkt, daarna wordt de vorm en werking van de 'main-app' uitgelegd. Ten slotte wordt de vorm en werking van de drie afzonderlijke apps beschreven.

Als eerst zijn er *mockups* gemaakt van de '*main-app*' en de andere drie *apps,* zodat er een visueel overzicht is van de verschillende *apps*.

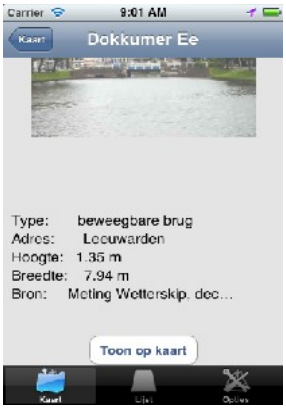
De drie apps worden gemaakt met een programma, genaamd: *Titanium Studio*. Er wordt door middel van *JavaScript* een '*main-app*' geprogrammeerd die als raamwerk dient voor de andere apps, die daarna worden geprogrammeerd.

4.2.1 De ‘main-app’

In deze *paragraaf wordt de vorm en werking van de ‘main-app’ beschreven.*

Afbeelding 5: Beginpagina

Afbeelding 4: Lijst



Afbeelding 6: Detail scherm

Afbeelding 7: Instellingen

Op de beginpagina wordt de kaart getoond met daarop de huidige positie en de *annotations* van de verschillende objecten. Hier kan op geklikt worden om extra informatie te krijgen over het aangeklikte object. Dit is hetzelfde scherm als bij de tweede pagina wanneer je een object aanklikt, de informatie hoort bij de annotatie die je op dat moment aanklikt. Bij *iOS* zit er onderin en bij *Android* zit er bovenin een tabbalk, waarmee je tussen de verschillende pagina’s kunt navigeren.

Op de tweede pagina staat een lijst met daarop, weer afhankelijk van de *app*, bruggen, aanlegplaatsen, visstekken etc. geordend op alfatbet. Als er in de lijst op een object wordt geklikt wordt het detailscherm geopend, met daarin alle gegevens over het aangeklikte object, waaronder de hoogte en breedte en een afbeelding van het object. Verder is er nog de ‘Toon op kaart’ knop waarmee het huidige object op de kaart wordt getoond. Met de zoekbalk op deze pagina kan de lijst met objecten worden doorzocht.

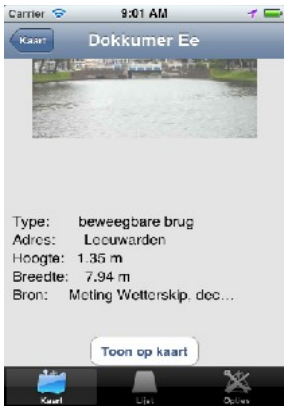
Bij het laatste scherm staan alle instellingen die nodig zijn om de *app* goed te laten functioneren. Zo kunnen bijvoorbeeld de hoogte en de breedte van de boot ingevuld worden. Nadat dit is ingevuld of gewijzigd wordt er ‘achter de schermen’ een *functie* aangeroepen die er voor zorgt dat de hoogte van de brug wordt vergeleken met de hoogte van de boot, hierdoor worden er rode of groene *annotations* op de kaart getoond, afhankelijk van de uitkomst. Bij de instellingen kan men een ander kaartype kiezen, het laden van externe afbeeldingen uitschakelen en is er een knop waarmee men naar de pagina met informatie over de *app* kan.

4.2.2 Bruggen en Aanlegplaatsen app

In deze paragraaf wordt de vorm en werking van de Bruggen en Aanlegplaatsen app beschreven.

Afbeelding 8: Beginscherm

Afbeelding 9: Lijst



Afbeelding 10: Detail scherm

Afbeelding 11: Instellingen

Op de beginpagina is een kaart te zien met daarop de huidige positie en de *annotations* van de bruggen en aanlegplaatsen in Friesland. Zodra hier op geklikt wordt krijg je een schermpje waarop de naam, hoogte en breedte van de brug staat of de naam van de aanlegplaats. Met de tabbalk kun je navigeren tussen de verschillende pagina's.

Op de tweede pagina krijg je een lijst met alle bruggen en aanlegplaatsen in Friesland te zien, geordend op afstand. Wanneer er op een brug of aanlegplaats wordt geklikt wordt het detailscherm geopend, met daarin alle gegevens over de brug of aanlegplaats, waaronder de hoogte en breedte en een afbeelding van het object (wanneer deze aanwezig is). Verder is er nog de 'Toon op kaart' knop waarmee de brug of aanlegplaats op de kaart wordt getoond. Met de zoekbalk op deze pagina kan de lijst met objecten worden doorzocht.

Bij het laatste scherm staan alle instellingen die nodig zijn om de app goed te laten functioneren. Zo kunnen bijvoorbeeld de hoogte en de breedte van de boot ingevuld worden. Nadat dit is ingevuld of gewijzigd wordt er 'achter de schermen' een *functie* aangeroepen die er voor zorgt dat de hoogte van de brug wordt vergeleken met de hoogte van de boot, hierdoor worden er rode of groene *annotations* op de kaart getoond, afhankelijk van de uitkomst. Bij de instellingen kan men een ander kaartype kiezen, het laden van externe afbeeldingen uitschakelen en is er een knop waarmee men naar de pagina met informatie over de app kan.

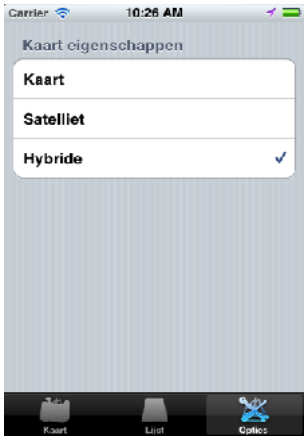
4.2.3 Bezienswaardigheden Leeuwarden app

In deze paragraaf wordt de vorm en werking van de Bezienswaardigheden Leeuwarden app beschreven.

Afbeelding 15: Beginscherm

Afbeelding 13: Lijst

Afbeelding 14: Detail scherm



Afbeelding 12: Instellingen

Op de beginpagina is een kaart te zien met daarop de huidige positie en de *annotations* van de bruggen, aanlegplaatsen en bezienswaardigheden in Leeuwarden. Zodra hier op geklikt wordt, krijg je een schermpje met informatie over de desbetreffende annotation. Met de tabbalk kun je navigeren tussen de verschillende pagina's.

Op de tweede pagina staat een lijst waarin de bruggen, aanlegplaatsen en bezienswaardigheden staan, geordend op afstand. Wanneer er op één van de genoemde objecten wordt geklikt, wordt het detailscherm geopend, met daarin alle gegevens over het desbetreffende object, waaronder de hoogte en breedte en een afbeelding van het object (wanneer deze aanwezig is). Verder is er nog de 'Toon op kaart' knop waarmee de brug of aanlegplaats op de kaart wordt getoond. Met de zoekbalk op deze pagina kan de lijst met objecten worden doorzocht.

Bij het laatste scherm staan alle instellingen die nodig zijn om de app goed te laten functioneren. Zo kunnen bijvoorbeeld de hoogte en de breedte van de boot ingevuld worden. Nadat dit is ingevuld of gewijzigd wordt er 'achter de schermen' een *functie* aangeroepen die er voor zorgt dat de hoogte van de brug wordt vergeleken met de hoogte van de boot, hierdoor worden er rode of groene *annotations* op de kaart getoond, afhankelijk van de uitkomst. Bij de instellingen kan men een ander kaartype kiezen, het laden van externe afbeeldingen uitschakelen en is er een knop waarmee men naar de pagina met informatie over de app kan.

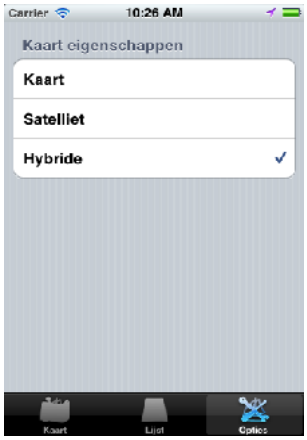
4.2.4 Visstekken app

In deze paragraaf wordt de vorm en werking van de Visstekken app beschreven.

Afbeelding 19: Beginscherm

Afbeelding 18: Lijst

Afbeelding 16: Detailscherm



Afbeelding 17: Instellingen

Op de beginpagina is een kaart te zien met daarop de huidige positie en de *annotations* van de visstekken in Friesland. Zodra hier op geklikt wordt, krijg je een schermpje met informatie over de desbetreffende *annotation*. Met de tabbalk kun je navigeren tussen de verschillende pagina's.

Op de tweede pagina staat een lijst waarin visstekken staan, geordend op afstand. Wanneer er op een visstek wordt geklikt, wordt het detailscherm geopend, met daarin alle gegevens over de desbetreffende visstek. Verder is er nog de 'Toon op kaart' knop waarmee de brug of aanlegplaats op de kaart wordt getoond. Met de zoekbalk op deze pagina kan de lijst met objecten worden doorzocht.

Bij het laatste scherm staan alle instellingen die nodig zijn om de *app* goed te laten functioneren. Zo kunnen bijvoorbeeld de hoogte en de breedte van de boot ingevuld worden. Nadat dit is ingevuld of gewijzigd wordt er 'achter de schermen' een *functie* aangeroepen die er voor zorgt dat de hoogte van de brug wordt vergeleken met de hoogte van de boot, hierdoor worden er rode of groene *annotations* op de kaart getoond, afhankelijk van de uitkomst. Bij de instellingen kan men een ander kaartype kiezen, het laden van externe afbeeldingen uitschakelen en is er een knop waarmee men naar de pagina met informatie over de *app* kan.

5 Het omzetten van data

In dit hoofdstuk word uitgelegd hoe de dataconvert tool is gemaakt en hoe die werkt.

Aan het begin van het project werdt de data aangeleverd in de vorm van een spreadsheet. Deze vorm is niet handig om te gebruiken binnen een applicatie, daarom is er voor gekozen om de data, voordat het meegeleverd wordt in de applicaties, om te zetten in een gebruiksvriendelijker formaat, namelijk *JSON.*

Omdat het niet makelijk is om met een simpel programmatje uit een Microsoft Excel spreadsheet bestand te lezen, is er voor gekozen om dat gebruiker van de *dataconverter* eerst zelf handmatig de spreadsheet via Microsoft Excel te laten exporteren naar *CSV* formaat. *CSV* is namelijk opgemaakt als platte text, waardoor het vele malen simpeler word om de data uit het bestand te lezen.

Het omzetten van *CSV* data naar *JSON* data is niet moeilijk, in de geexporteerde *CSV* data staan alle data objecten netjes op een regel. Het is daardoor mogelijk om de data regel voor regel uit te lezen en vervolgens elk dataobject om te zetten in een *JSON* met behulp van een stukje software. Vervolgens hoeft de data alleen nog maar in een *JSON* array verpakt te worden en geschreven naar een bestand.

Het mooie van het systeem met een spreadsheet is dat je heel makkelijk de data die je wilt gebruiken kunt samenstellen. Je kunt namelijk heel makkelijk in een programma als Microsoft Excel de namen van de kolommen in de spreadsheet aanpassen en ook kun je makkelijk een kolom uit de *dataset* halen. Dit maakt het heel makkelijk om de data geschikt te maken voor de apps die zijn ontwikkeld.

Helaas moest de spreadsheet halverwege het project plaats maken voor een andere manier van aanleveren van data, namelijk het gebruik van het Provinciaal Georegister. De keuze hiervoor kwam vanuit de praktische reden, dat deze set data onderhouden word door de provincie, wat niet het geval was met de data in de spreadsheet. Het Provinciaal Georegister levert zijn data als een *GML* bestand, *GML* is een dialect van *XML.*

```
– <food>
    <name>Belgian Waffles</name>
    <price>$5.95</price>
  – <description>
      two of our famous Belgian Waffles
    </description>
    <calories>650</calories>
  </food>
– <food>
    <name>Strawberry Belgian Waffles</name>
    <price>$7.95</price>
  – <description>
      light Belgian waffles covered with strawberries
```

Afbeelding 21: Een kort stukje XML

```
{
    hey: "guy",
    anumber: 243,
  - anobject: {
        whoa: "nuts",
        anarray: [
            1,
            2,
            "thr<h1>ee"
        ],
        more: "stuff"
    },
    awesome: true,
    bogus: false,
    meaning: null,
    japanese: "明日がある。",
    link: http://jsonview.com,
    notLink: "http://jsonview.com is great"
}
```

Afbeelding 20: Een kort stukje JSON

Met *GML* verlies je wel het voordeel dat je voor het omzetten de data nog even makkelijk kan aanpassen. Eerst was er nog het plan om deze functionaliteit in te bouwen in de *dataconverter*, maar gezien dat iets te veel ontwikkeltijd ging kosten is dat plan geschrapt. In plaats daarvan is er voor gekozen om in de software zelf al van te voren aan te geven welke kolommen gebruikt en/of hernoemd moesten worden. Je verliest hierdoor een stuk flexibiliteit, want als er een nieuwe of veranderde set data komt, dan zal iemand deze wijzigingen in de broncode van het programma moeten doorvoeren.

Het uiteindelijk gemaakte data convert programma is geschreven in de *Java* programmeertaal. Hiervoor is gekozen, omdat *Java* beschikt over ingebouwde functionaliteit om *GML* in te lezen. Hierdoor hoeft er voor het programma alleen nog maar functionaliteit geschreven te worden voor het aanpassen van de inhoud van de data en het schrijven naar een bestand als *JSON* data.

Het gebruik van het data convert programma is niet heel erg moeilijk, het wordt geleverd als een *Java Jar* bestand. Deze kun je simpelweg via de command line van de computer aanroepen en daarbij het bestandsnaam van het *GML* bestand dat je wilt omzetten aangeven. Het programma neemt dan het bron *GML* bestand en maakt op de zelfde locatie een *JSON* bestand met dezelfde naam aan, waarin de omgezette data word geschreven, deze data is vervolgens direct bruikbaar in de applicatie.

6 Verloop van het project

In dit hoofdstuk wordt besproken hoe het project verlopen is en welke obstakels er tijdens het project aan het licht zijn gekomen.

Gedurende het project is de opdracht een aantal keren veranderd. Dit was soms niet geheel zonder tijdverlies. Zo moest er aanvankelijk, tijdens onze eerste ontmoeting met de opdrachtgever, één app gemaakt worden. Echter in de week daarna moest er opeens vier verschillende apps komen met elk een iets andere *dataset*. Na een verder overleg met de opdrachtgever is er tot de conclusie gekomen dat twee van de vier verschillende apps eigenlijk beter samengevoegd kunnen worden. Hierdoor werd het aantal apps dat gemaakt moet worden verlaagd naar drie. Bij het maken van de apps was het de bedoeling dat er data over verschillende objecten in moest komen, hierdoor kwam er een andere opdracht bij namelijk: het maken van een *dataconverter*. Marten zag deze opdracht wel zitten en is hier mee bezig geweest. Deze taak was aanvankelijk bijna af, totdat de opdrachtgever een andere *dataset* wou gebruiken welke via de overheid wordt aangeleverd. Het gebruik van deze *dataset* is iets lastiger, omdat deze anders in elkaar zit.

De laatst genoemde *dataset* heet *GML* (Geography Markup Language). Deze *dataset* is gebruikt om punten in te laden voor de apps. Voordat dit in gebruik kan worden genomen moet dit wel eerst omgezet worden naar iets wat de app kan gebruiken met een zo genaamde *dataconverter.* Het *GML* bestand zit echter vrij lastig in elkaar en is veel lastiger dan het omzetten van een excel bestand, zoals eerst de bedoeling was.

Halverwege het project bleek dat de opbouw van onze code slecht was. De gegevens op verschillende pagina’s konden niet worden aangepast vanaf een andere pagina. Dit had als gevolg dat de hele code omgegooid moest worden naar een andere vorm. Dit leverde een paar dagen vertraging op.

In de laatste weken van het project konden we testen met de volledige *dataset.* Toen kwam aan het licht dat de manier waarop er punten op de kaart getoond werden niet meer voldeed. Alle punten konden niet meer op de kaart weergegeven worden. Dit was te langzaam. Er is toen een zoomlevel bepaald en vanaf dat zoomlevel kwamen er punten op de kaart. Dit had als gevolg dat wanneer de kaart veranderd de punten op de kaart gaan knipperen. Dit is een beperking van *Titanium Studio*, hier is niets aan te veranderen zolang er *Titanium Studio* gebruikt word. Andere opties waren nog minder soepel dan die werd gebruikt.

7 Samenwerking

In dit hoofdstuk bespreken we de samenwerking binnen onze groep en die met de opdrachtgever.

De samenwerking binnen de groep was erg goed. We waren alle dagen samen aan het werk in het hokje van de opdrachtgever. Iedereen was er doorgaans om een uur of negen en wanneer iemand van ons niet kon werd dit op tijd doorgegeven. We stopten in de regel rond een uur of 3 a vier. In de tussentijd werd er veel overlegd over wat er komen moest en ook op welke manier. Dankzij het gebruik van GitHub als VCS konden we allemaal aan ons eigenstukje werken zonder dat we op een ander hoefden te wachten. Dit was erg handig.

De samenwerking met de opdrachtgever was ook wel aardig goed. De opdrachtgever heeft zelf een studie informatica afgerond en dit had als voordeel dat we toch iets technischer konden praten. Het was alleen wel iets lastiger om een afspraak met de opdrachtgever te krijgen. Die kwam niet altijd opdagen op afspraken en soms werden ze ook niet nageleefd.

8 Conclusies

In dit hoofdstuk worden de conclusies beschreven en worden enkele aanbevelingen gegeven.

Om een app gebruiksvriendelijk te maken zijn er een aantal dingen waar rekening mee moet worden gehouden:

1. De app moet makkelijk sluiten.
2. De gegevens moeten zo weinig mogelijk intern geheugen in beslag nemen.
3. De app moet overzichtelijk zijn, zodat het navigeren vlot en gemakkelijk gaat.
4. De app moet snel zijn, mensen houden nou eenmaal niet van wachten.
5. Zorg ervoor dat, wanneer de app op de achtergrond draait, er niet teveel energie wordt gebruikt.
6. Probeer zo volledig mogelijk de mogelijkheden van de telefoon te benutten.
7. De app moet de gebruiker feedback geven.
8. De bediening moet simpel en snel aan te leren zijn.

De grootste verschillen tussen *iOS* en *Android* zijn:

- *iOS* heeft niet de mogelijkheid, in tegenstelling tot *Android*, om de telefoon helemaal te personaliseren.
- *iOS* heeft maar één type telefoon en *tablet* van hetzelfde merk. *Android* heeft daarentegen meerdere types telefoons en *tablets* van verschillende merken.

De applicatie wordt goed werkend op beide platformen, door een goede ontwikkelomgeving te kiezen, waarin er gemakkelijk onderscheid kan worden gemaakt tussen *iOS* en *Android*.

De ontwikkelomgeving die gekozen is: *Titanium Studio.* Het grootste voordeel hiervan is, dat je snel en *cross platform*, *native apps* kunt ontwikkelen, met een populaire programmeertaal, genaamd *JavaScript*.

Om de punten op de kaart te krijgen, zijn er de ingebouwde *annotations* van *Google Maps*. Om ze overzichtelijk op een scherm te laten zien is er de mogelijkheid om zodra je uitzoomt op een kaart de *annotations* te groeperen in hoeveelheden.

De huidige positie wordt bepaald door middel van *GPS*, hiermee wordt via satellieten de huidige positie bepaald. Om hierna een *trail* op de kaart te tonen is niet lastig. De huidige positie wordt onthouden en zodra deze zich verplaatst wordt er een stip op de kaart getoond die de *trail* vormt

De straal die het beste kan worden gebruikt voor het tonen van de kaart, zodra de gebruiker de app opent is de gehele regio waar het om gaat, mits er geen *GPS* verbinding is.

De beste manier om een kaart in de app te krijgen is met *Google Maps.* Deze kaart is online te gebruiken door middel van mobiel internet, *Wi-Fi* en *GPS.* Ook is de kaart offline te gebruiken, want die *cached* het gedeelte van de kaart waar de gebruiker naar kijkt.

9 Aanbevelingen

In dit hoofdstuk doen we aanbevelingen voor het vervolgen dan wel verder uitwerken van de apps

9.1. Design appicoon

In deze paragraaf beschrijven we hoe het icoontje van de app het beste kan worden ontworpen.

De *App Store* van apple wil 10 verschillende appicoongroottes krijgen. Het is veel werk om deze allemaal op de juiste schaal op te slaan. Hiervoor is een *template* te vinden waardoor je maar 1 design/maat hoeft te maken en de rest is te doen met '*Photoshop* actions'. Dit zorgt dat alles in de juiste naamgeving en in de juiste afmetingen opgeslagen wordt. Deze *template* is te vinden en te downloaden op http://appicontemplate.com. Het is aan te raden om deze te gebruiken gezien dit een hoop werk bespaard.

9.2 Design iconen in de app

In deze paragraaf wordt beschreven wat de eisen zijn voor een icoon binnen de applicatie.

Alle plaatjes binnen de app moeten op ware schaal gemaakt worden, wanneer dit te klein wordt gemaakt moet het programma dit zelf vergroten. Zodra dit gebeurt gaat de kwaliteit van de plaatjes er op achteruit. Andersom is dit vaak niet zo, maar bij *Titanium Studio* wel. Wanneer een afbeelding kleiner afgebeeld moet worden dan hij werkelijk is gaat de kwaliteit er ook op achteruit. Daarom is het op schaal maken van een plaatje aan te raden om de beste kwaliteit te garanderen.

9.3 Titanium Studio

In deze paragraaf wordt uitgelegd waarom er afgeraden wordt Titanium Studio te gebruiken voor een soortgelijks project.

Allereerst is de documentatie slecht. Bij de online documentatie van *Titanium Studio* staat er vrij weinig informatie en uitleg over de genoemde punten. Dit maakte het erg lastig in het begin om tussen de juiste onderdelen te kiezen en om deze juist te kunnen gebruiken.

Verder werkt *Titanium Studio* door een '*JavaScript* Engine' te draaien op de telefoon. Dit heeft als voordeel dat het op meerdere platformen werkt, maar als nadeel dat het minder snel draait. Dit had ook tot gevolg dat de app niet goed overweg kon met alle 3000 punten op de kaart. Er waren nog verschillende manieren bedacht om het inladen van de punten op de kaart te versoepelen, maar dit was niet haalbaar om dit binnen *Titanium Studio* te doen. Ook is het moeilijker is om fouten te vinden in de app, doordat het gebruik maakt van een '*JavaScript* Engine'. Hierdoor moest er, door middel van een omweg, getest worden. Ook dit leverde enig tijdsverlies op.

Tenslotte is het niet gelukt om alle wensen te verwezenlijken. Zo was er ook nog een idee om de lijst in de app te sorteren op afstand tussen de gebruiker en het opgezochte punt. Ook dit was niet haalbaar binnen *Titanium Studio.* Dit was namelijk veel te zwaar om op zo'n manier te gebruiken.

Kortom, voor een vervolg of een soortgelijk project is het beter om native te gaan werken. Dit geeft meer vrijheid om dingen te verwezenlijken en draait sneller.

Verklarende Lijst

Alle moeilijke woorden in de tekst, deze zijn meestal cursief, zijn hier alfabetisch geordend met een verhelderende uitleg erbij. [8]

Android: Een besturingssysteem voor mobiele telefoons gemaakt door Google.
Android Play Store: Een online softwarewinkel van Google waar software gekocht kan worden. De software, ook wel apps genoemd, omvat programma's voor de Android smartphone en tablet.
Annotations: Dit zijn punten die op de kaart van Google Maps worden getoond.
App Store: Een app Store is een online softwarewinkel van apple waar software gekocht kan worden. De software, ook wel apps genoemd, omvat programma`s voor de iPhone en iPad.
Cached: Geheugen van de telefoon dat gereserveerd is voor eerder geladen apps. De inhoud van deze apps worden door de telefoon onthouden om ze sneller te laten verschijnen op het moment dat de gebruiker ze opnieuw opvraagt.
Cross Platform: Een (software)programma dat gebruikt kan worden op meerdere computertypes.
CSV: Afkorting voor comma seperated values, een specificatie om tabeldata op te slaan als platte text.
Dataconverter: Een programma wat de weergave van gegevens in een database van één vorm naar een andere verandert, zoals bijvoorbeeld door verandering van opslagmedium, gegevensindeling of de code waarin de gegevens zijn opgeslagen.
Dataset: Een collectie van data, gegevens, meestal gepresenteerd in tabelvorm.
Functie: Een onderdeel van een programma dat losstaat van de rest van het programma. Een functie kan worden opgeroepen vanuit het programma. Functies vervullen meestal een specifieke taak. Je kunt ze zien als hele kleine programma's of scripts.
GML: Afkorting voor geography markup language. Dit is een dialect van XML, zie XML.
Google Maps: Een applicatie waarin gezocht kan worden naar locaties over de hele wereld.
GPS: Global Positioning System, systeem waarbij met behulp van satellieten exact de positie wordt bepaald waar men zich bevindt (o.a. gebruikt op schepen en in auto`s).
iOS: Een besturingssysteem voor mobiele telefoons gemaakt door Apple.
iPad: De iPad is een tablet-pc van het Amerikaanse elektronicabedrijf Apple. Net zoals de iPhone en de iPod Touch bevat de iPad een multi-touchscherm, waardoor deze met de vingers is te bedienen.
iPhone: Dit apparaat was met de HTC Touch een van de eerste smartphones met een on-screen-toetsenbord. Het heeft geen mechanische toetsen om telefoonnummers of sms-berichten in te tikken. De iPhone gebruikt iOS, een aangepaste versie van Mac OS X (het besturingssysteem van Apple).
iPod Touch: een draagbare mediaspeler uit de iPod serie van Apple, voorzien van een multitouchscherm.
Jar: Java Archive, is het standaard formaat om een Java project te archiveren waardoor het makkelijker te gebruiken is.
Java: Een platform onafhankelijk objectgeoriënteerde programmeertaal.
JavaScript: Een objectgeoriënteerde programmeertaal die veel gebruikt wordt om internetpagina`s interactief te maken en webapplicaties te ontwikkelen.
JSON: Afkorting voor JavaScript object notation, wordt gebruikt om data op een eenvoudige

manier als text op te slaan.
Main-app: De applicatie die dient als raamwerk voor alle drie de apps.
Mockup: Een tekening/afbeelding op schaal van, in dit geval, een app voor op een mobiele telefoon of tablet.
Native app: Een programma dat specifiek ontwikkeld en geïmplementeerd is voor het gegeven model, microprocessor of besturingssysteem.
Photoshop: Professionele software van Adobe voor het bewerken en maken van afbeeldingen.
Photoshop actions: Voorgeprogrammeerde acties in Photoshop.
Smartphone: Een smartphone is een mobiele telefoon die uitgebreidere computermogelijkheden biedt. Een smartphone kan ook beschouwd worden als een handcomputer of pda die tegelijk ook een telefoon is.
Tablet: Computer die geen toetsenbord of muis heeft, maar enkel werkt met een touchscreen.
Template: Sjabloon waarin de vorm van een pagina is vastgelegd. Het gebruik van templates garandeert dat content en vormgeving van elkaar worden gescheiden.
Titanium Studio: Een softwareontwikkelomgeving van appcelerator, waarmee apps kunnen worden ontwikkeld voor de smartphone, tablet, desktop en het web.
Trail: Een spoor van je huidig afgelegde route, zodat je altijd weet in welke richting je aan het varen bent.
Widgets: Een klein programmaatje dat als een soort van gadget werkt.
Wi-Fi: Techniek waarmee een computernetwerk draadloos met behulp van een radiozender en radio-ontvanger kan worden gebouwd. Er zijn geen kabels nodig in dit netwerk, maar slechts een insteekkaart in een computer.
XML: Afkorting voor Extensible Markup Language. Een opmaaktaal waarmee gegevens gestructureerd kunnen worden weergegeven in de vorm van platte tekst.

Bronnenlijst

[1] Android Planet – http://www.androidplanet.nl/android-tips/basisfuncties/10-tips-voor-succesvolle-android-applicaties/

[2] Wikipedia – Gebruiksvriendelijkheid – n*l.wikipedia.org/wiki/Gebruiksvriendelijkheid -- Bezocht: 20-04-2012*

[3] Consumentenbond – Mobiele-besturingssystemen -- *http://www.consumentenbond.nl/test/elektronica-communicatie/telefonie/mobiele-telefoons/extra/mobiele-besturingssystemen/ -- Bezocht: 20-04-2012*

[4] Titanium Appcelerator – *http://www.appcelerator.com/platform/titanium-studio -- Bezocht: 18-04-2012*

[5] Google Maps – *http://maps.google.nl/maps?hl=nl&tab=ll -- Bezocht: 27-04-2012*

[6] Google Developer Center – Maps – *https://developers.google.com/maps/ -- Bezocht: 27-04-2012*

[7] Appcelerator Wiki – Tracking Position and Heading – *https://wiki.appcelerator.org/display/guides/Tracking+Position+and+Heading -- Bezocht: 08-05-2012*

[8] Encyclo – *http:// www.encyclo.nl – Bezocht: 08-05-2012*

Afbeeldingen:

1: http://dailymobile.se/wp-content/uploads/2012/06/android-logo.png
2: http://edibleapple.com/wp-content/uploads/2009/04/apple_rainbow_logo.jpeg
20: http://jsonview.com/images/jsonviewscreen-8e428fa0.png
21: *http://www.urbaninsight.com/images/articles/xmarks.gif*

Bijlagen

A Keuze ontwikkelplatform voor water apps

Voordat je apps gaat maken, moet je het eerst wel eens worden over een platform waarmee je het gaat ontwikkelen. Daarbij houd je rekening met de specificaties van apps, voor ons waren de belangrijkste punten:

- De app moet voor meerdere platformen gemaakt worden. (Minstens Android en iOS)
- De ontwikkelomgeving moet ook cross platform zijn. (Windows, OS X, Linux)
- Google maps api met offline caching.
- Liefst geen kosten.

Aan de hand van de bovenstaande eisen hebben we een shortlist gemaakt met ontwikkel platformen die ons interressant leken. Dit zijn:

- Phonegap / Apache Cordova
- Titanium Appcelerator
- MonoTouch & MonoDroid
- De native ontwikkel omgeving van elk smartphone platform

Titanium Appcelerator:
Als eerste hebben we Titanium Appcelerator onderzocht, het grote voordeel van dit platform is dat je snel en cross platform native apps kunt ontwikkelen, met een programmeertaal die erg populair is.

Het platform ondersteunt in zekere zin Android, iOS en Blackberry. Voor iOS geldt altijd dat je alleen op een OS X pc kunt testen. Bij Titanium appcelerator is ook nog het geval dat je alleen voor Blackberry kunt testen op een Windows pc. Het is dus wel cross platform, maar toch enigzins beperkt, maar het voldoet wel aan de eis.

apps ontwikkelen met appcelerator doe je met de JavaScript programmeertaal, het gebruik van JavaScript hierin is een combinatie van een soort van declaratief programmeren, in het geval van user interface elementen en object georienteerd programmeren in bij het maken van het programma achter de app zelf.

Het platform maakt gebruik van het smartphone eigen user interface elementen en heeft ook toegang tot (bijna) alle api's die het smartphone platform ondersteunt. Daarbij hoort dus ook de Google maps api.

Een nadeel van appcelerator is het eigen ontwikkelomgeving (Titanium Studio), de installatie hiervan is niet echt vanzelfsprekend en ook documentatie voor de installatie ontbreekt grotendeels. Alleen op OS X werkte Titanium Studio gelijk na de intallatie. Bij de installatie

met Windows kon het programma zelf niet de Java JDK vinden en moesten er een aantal onderliggende omgevings variabelen aangepast worden zodat het programma wel het Java JDK kon vinden. De installatie op Linux was ook niet makkelijk, je krijgt een gecomprimeerd bestandje met de Studio voorgeinstalleerd, als je deze opstart kreeg je een aantal fout meldingen van missende libraries en bestanden. Uiteindelijk moest er eerst nog het volgende geinstalleerd worden om het programma werkend te krijgen in Linux:

- De proprietaire Oracle Java JDK, die officieel niet door Linux ondersteund wordt.
- Een extra SDK van Titanium zelf.
- De libpng62 library.

Als voetnoot willen we er nog bij zeggen dat Titanium Studio eigenlijk niet veel meer is dan een mix-up van Eclipse, het ontwikkelplatform voor o.a. native Android apps en Java applicaties, is.

De eerste keer dat we naar dit platform hadden gekeken, hebben we het al snel aan de kant gezet, omdat we Titanium Studio na een paar uur debuggen nog niet werkend hadden.

Native
Het volgende platform wat we na Titanium studio hebben bekeken is het platform waar het meeste werk in gaat zitten, dat is namelijk het gebruik van het eigen app ontwikkelplatform van elk apart smartphone platform, als we dan dus aan onze minimale cross platform eis willen voldoen komt dat neer op het gebruik van xCode voor iOS apps en Eclipse voor Android apps, je bent dan met twee verschillende programmeertalen aan het werk, repectievelijk Objective-C en Java. Je moet dan dus in principe elke app twee keer schrijven en twee keer onderhouden. Daarnaast is het ontwikkelen van een specifieke iOS app ook alleen maar mogelijk met een OS X pc.

Door de bovengenoemde redenen is native dus geen goede keuze. Het voldoet niet goed aan onze gestelde eisen.

Phonegap / Apache Cordova
Als derde hebben we Phonegap getest, dit platform is uit alle mogelijkheden die we hebben onderzocht het meest cross platform, het ondersteund Android, iOS, Blackberry, Windows Phone, Symbian en webOS.

Het platform heeft geen eigen ontwikkelomgeving, in plaats daarvan is er op website van Phonegap een handleiding hoe je in een korte tijd voor elk platform een template project opzet, vervolgens is er een map 'www' die je app code bevat, deze deel je tussen de template projecten van de verschillende projecten.

De app zelf maak je met HTML, CSS en JavaScript, dit werkt precies zoals je een normale website voor een smartphone zou maken en is dus vrij makelijk te doen, ook is zijn er bij dit platform doormiddel van een JavaScript library de meeste native api's beschikbaar. Helaas een van de weinige api's die dan net weer niet beschikbaar is, is die van Google Maps. Hierdoor zouden we de JavaScript Google maps api moeten gebruiken, waarvan de

performance op het mobiele netwerk zeer ondermaats is.

Omdat het met HTML, CSS en JavaScript werkt, zijn er ook geen native user interface elementen beschikbaar, deze zouden we dan dus zelf moeten ontwikkelen, of een 3rd party library, zoals jQuery Mobile, daarvoor gebruiken.

Ook al leek het ons op het eerste gezicht een goed platform, toch moesten we het voor dit project laten varen, omdat het niet aan onze Google Maps eis voldoet.

MonoTouch / MonoDroid
Last but not least, hebben we MonoTouch/MonoDroid onderzocht, dit zijn twee verschillende platformen van hetzelfde bedrijf. MonoTouch is voor iOS en MonoDroid is voor Android. Het voldoet dus aan de minimale eis van cross platform zijn.

Het platform werkt met C#, het grote voordeel hiervan is dat alle groepsleden al vaker met C# gewerkt hebben. Ook belooft het platform, dat je ondanks dat het twee aparte platformen zijn, toch vrij veel van de app code onder elkaar kunt uitwisselen. Daarnaast is alles native en heb je dus beschikking tot de native user interfaces, native api, waaronder dus ook de Google Maps api.

Het ontwikkelen met MonoTouch en MonoDroid doe je met Microsoft Visual Studio of MonoDevelop, beide zijn goed werkende programmas waar goed in te ontwikkelen valt.

Helaas moet ook dit platform afvallen omdat het niet aan het gewoonweg te duur ($399) is om er mee te ontwikkelen.

Conclusie
Uiteindelijk zijn we weer terug gekomen op Titanium appcelerator, na een aantal extra uren debuggen hebben we Titanium Studio op elke pc aan het werk gekregen en is het gelukt om een 'Hello World' app werkend te krijgen. De apps gaan dus ontwikkeld worden in Titanium Studio.

Als we een andere app zouden ontwikkelen waar we niet verplicht de Google Maps api nodig hadden, dan hadden we voor Phonegap gekozen, het gebruik daarvan is een stuk makkelijker dan bijvoorbeeld Titanium Appcelerator.